Summary Report

Volleyball-Group-Activity-Recognition

Omar Ayman Tawfiq Bakr

Abstract—This report documents the data pipeline, loader architecture, and first experimental results for the Volleyball Group Activity Recognition project. The dataset, introduced in the CVPR 2016 paper by Mostafa S. Ibrahim et al., contains 55 volleyball videos with two annotation levels: 8 scene-level group activities and 9 player-level individual actions. We describe the two-stage parsing pipeline that unifies three raw annotation sources into a single fast-loading pickle cache, and the generic PyTorch data loader that supports all eight baseline models (B1–B8). We also present Baseline 1 (B1), a two-stage fine-tuned ResNet-50 single-frame classifier, including its training configuration and test-set evaluation.

Keywords— *Volleyball, Group Activity Recognition, CNN, RNN, LSTM, ResNet50, Classification, Multiclass Classification, Transfer Learning*

1. Introduction

Volleyball Group Activity Recognition is a hierarchical classification problem where the goal is to classify the group activity of a volleyball scene (one of 8 classes) while also recognising individual player actions (one of 9 classes). The dataset was introduced in the CVPR 2016 paper "A Hierarchical Deep Temporal Model for Group Activity Recognition" by Mostafa S. Ibrahim et al. You can download the dataset from the official repository or from Kaggle.

2. Dataset Summary

The dataset has **two levels of annotation**:

2.1. 9 Person Actions (Player-Level)

- Blocking
- Digging
- Falling
- Jumping
- Moving
- Setting
- Spiking
- Standing
- Waiting

The *standing* class is the most frequent action in the dataset, resulting in a significant class imbalance as shown in Fig. 1.

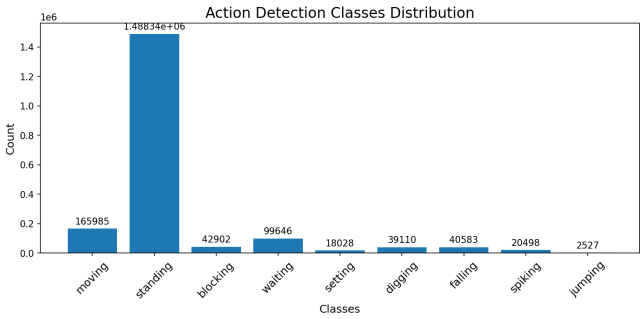

Figure 1. Distribution of person-action classes across all videos.

2.2. 8 Group Activities (Scene-Level)

- Left pass (`l-pass`)
- Right pass (`r-pass`)
- Left spike (`l-spike`)
- Right spike (`r_spike`)
- Left set (`l_set`)
- Right set (`r_set`)
- Left win-point (`l_winpoint`)
- Right win-point (`r_winpoint`)

The tracking-based action distribution is shown in Fig. 2.

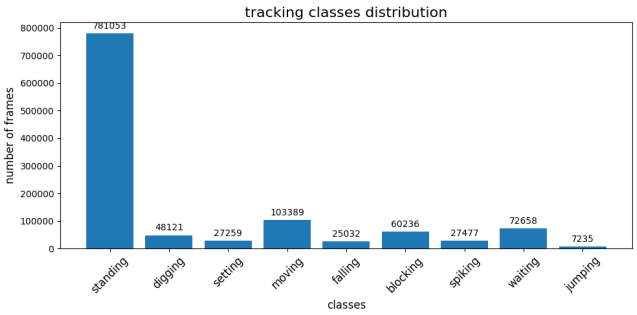

Figure 2. Distribution of tracking action classes across all videos.

2.3. Annotation Classes (from annotations.txt)

The `annotations.txt` file at the video level contains both group-activity labels and per-person action labels. Counting all labels across the 55 videos produces the combined distribution shown in Fig. 3. The *standing* class dominates with 38,686 occurrences, followed by *moving* (5,120) and *waiting* (3,600). Group-activity labels (`l-pass`, `r_spike`, etc.) appear once per clip, totalling 4,830 across the dataset.

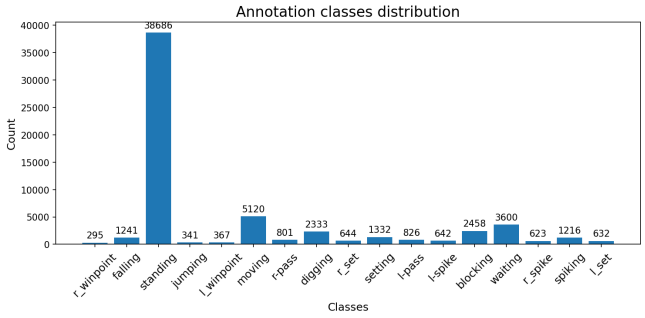

Figure 3. Distribution of all annotation labels (group activities + person actions) from annotations.txt.

2.4. Dataset Statistics

Table 1. Data split sizes

Split	Videos	Clips
Train	24	2,152
Validation	15	1,341
Test	16	1,337
Total	55	4,830

Each clip contains **41 consecutive frames**. The clip name is the middle frame number, which serves as the key frame for annotation.

3. Raw Data Sources

The raw dataset provides three separate annotation files per clip:

1. `action_detections.txt` — Tab-separated file with per-frame action detections. Each line contains the frame name, number of entries, and for each entry: bounding box (x, y, w, h), confidence score, and action label.

2. `person_detections.txt` — Same format as action detections but for generic person detection.
3. `clip_id.txt` (tracking) — Space-separated file with tracked players across frames. Each line: track ID, bounding box (x_1, y_1, x_2, y_2), frame number, three flag attributes, and action label.
4. `annotations.txt` (per video) — Contains the group activity label and per-person bounding boxes for each clip's middle frame.

4. Two-Stage Parsing Pipeline

We unify the raw annotations into a single data structure through a two-stage pipeline implemented in `src/json_parser.py`.

4.1. Stage 1: Player-Level Parsing

`create_master_json()` iterates over all 55 videos and their clips, parsing the three annotation files into a unified JSON structure keyed by `video_id/clip_id`. Each clip entry contains:

- `actions`: per-frame action detections {frame → list of {box, score, label}}
- `persons`: per-frame person detections (same format)
- `tracking`: per-frame tracked players {frame → list of {id, box, flags, action}}

4.2. Stage 2: Scene-Level Enrichment

`enrich_with_scene_labels()` reads each video's `annotations.txt` to extract the group-activity label and merges it into the master JSON under a new `scene_class` key per clip.

4.3. Pickle Caching

The enriched master JSON (∼1.6 GB) is dumped to a pickle file (∼247 MB) via `pickle_dump.py` for fast loading. The dump function is a **singleton**: it skips re-dumping if the pickle file already exists.

5. Data Loader Architecture

The `VolleyballDataset` class in `src/data/data_loader.py` is a generic PyTorch `Dataset` that loads from the pickle cache and supports all baseline models through constructor flags.

5.1. Constructor Parameters

- `mode`: Dataset split — `"train"`, `"validation"`, or `"test"`
- `n_frames`: Number of frames per clip (1 or 9). Must be a positive odd number.
- `full_image`: Return full-resolution frames (for B1, B4)
- `crop`: Return per-person crops from bounding boxes (for B3, B5–B8)
- `transform`: A torchvision transform pipeline

5.2. Return Shapes

Table 2. Data loader output shapes by configuration

Config	Baseline	Output
full, T=1	B1	(image, group_label)
full, T=9	B4	(images $[T, C, H, W]$, group_label)
crop, T=1	B3	(crops $[P, C, H, W]$, person_labels $[P]$, group_label)
crop, T=9	B5–B8	(crops $[T, P, C, H, W]$, person_labels $[P]$, group_label)

T: number of frames, P: number of players (up to 12), C: channels, H/W: spatial dimensions.

5.3. Collate Function

A custom `collate_fn` handles the variable number of players per clip by padding the player dimension to the batch maximum and returning a boolean mask tensor indicating valid player positions.

5.4. Person Boxes

For crops, the loader uses **tracking data** (which provides consistent player IDs across frames) as the primary source for bounding boxes, with a fallback to action detections. This is critical for temporal baselines (B5–B8) that need to track the same player across the 9-frame window.

6. Baseline Models Overview

The project implements a progressive series of baseline models, each adding complexity:

Table 3. Baseline model characteristics

Model	Temporal	Player	Scene
B1	—	—	Image clf.
B3	—	Crop clf.	Max-pool
B4	LSTM	—	LSTM
B5	LSTM/player	Max-pool	NN
B6	LSTM/image	Max-pool	LSTM
B7	LSTM_1 + LSTM_2	Max-pool	LSTM_2
B8	LSTM_1 + LSTM_2	Team-split	LSTM_2

- **B1**: Fine-tune ResNet50 on the middle frame as an 8-class image classifier.
- **B3**: Fine-tune ResNet50 on cropped persons (9 action classes), extract features, max-pool all players, train NN over 8 group classes.
- **B4**: Extract frame-level representations using B1's backbone over 9 frames, train LSTM on the temporal sequence.
- **B5**: LSTM on per-player temporal crops, max-pool player representations, classify with NN.
- **B6**: Pool person features per frame, then LSTM on the image-level sequence.
- **B7**: Full hierarchical model — LSTM_1 per player, max-pool per frame, LSTM_2 on the frame sequence.
- **B8**: Same as B7, but pools team 1 (6 players) and team 2 (6 players) separately, then concatenates.

7. Baseline 1: Single-Frame Classifier

7.1. Architecture

Baseline 1 fine-tunes a pretrained ResNet-50 on the middle frame of each clip to directly predict the 8-class group activity label. No temporal or player-level modelling is used; the final fully-connected layer is replaced with a linear head of size 8.

7.2. Training Strategy

Training follows a two-stage schedule to stabilise optimisation:

1. **Linear probe** — backbone frozen, head trained for 5 epochs at $\text{lr} = 10^{-3}$.
2. **Full fine-tune** — entire network unfrozen, up to 50 epochs with differential learning rates (backbone 10^{-4}, head 3×10^{-4}) and cosine annealing.

7.3. Test-Set Performance

7.4. Confusion Matrix

Fig. 4 shows the per-class prediction confusion on the test split. Rows are ground-truth classes; columns are predicted classes.

Table 4. Baseline 1 hyperparameters

Hyperparameter	Value
Backbone	ResNet-50 (pretrained)
Stage 1 epochs	5, lr $= 10^{-3}$
Stage 2 max epochs	50, $\text{lr}_{\text{backbone}} = 10^{-4}$
Head lr multiplier	3×
LR scheduler	CosineAnnealingLR
Label smoothing	0.1
Weight decay	0.05
Batch size	128
Early stopping	patience = 10 epochs

Table 5. Baseline 1 test-set metrics (*to be updated*)

Metric	Value
Accuracy	*TBD*
Macro F1	*TBD*
Test Loss	*TBD*

8. Project Structure

The codebase is organized as follows:

- `configs/` — Centralised configuration: paths, splits, label mappings, and per-baseline Hydra YAML configs.
- `src/json_parser.py` — Two-stage parsing pipeline.
- `src/pickle_dump.py` — Singleton pickle dump/load utility.
- `src/data/kaggle_data_loader.py` — Generic PyTorch Dataset and collate function.
- `src/data/data_summary.py` — Dataset statistics and class distributions.
- `models/baseline1.py` — B1: two-stage fine-tuned ResNet-50 (*complete*).
- `utils/utility.py` — Training/evaluation loop helpers.
- `utils/evaluate.py` — Post-training evaluation: loads checkpoint, runs inference, generates all metric plots.
- `utils/plotting.py` — Confusion matrix, classification report, PR curves, mAP bar chart.
- `utils/load_model_config.py` — Hydra config builders for transforms and scheduler.

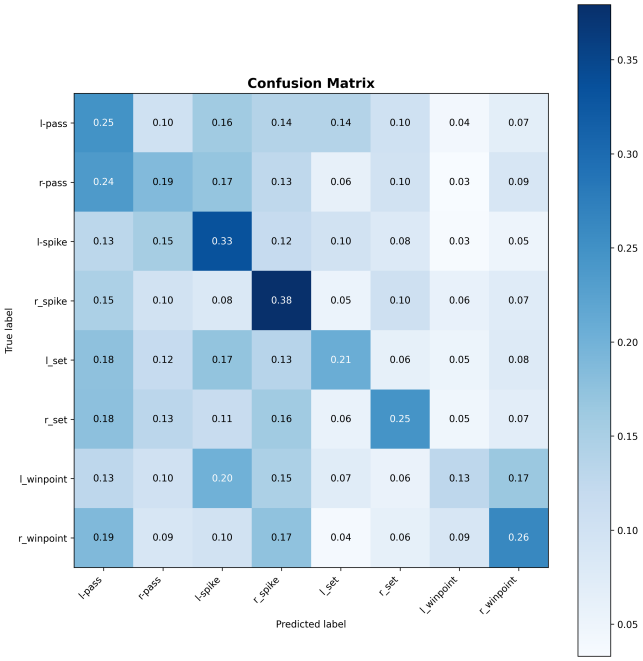

Figure 4. Confusion matrix for Baseline 1 on the test split.